UNIVERSITY CLUB MANAGEMENT PLATFORM (ClubMS)

B.Sc. Level 3 Semester II PROJECT REPORT
Course Code: CSE 352
Course Title: Web and Mobile Application Development Sessional

Prepared by

Md Farhan Quadery
ID: 2202015

Md Saddam Hosen Molla
ID: 2202042



Department of Computer Science and Engineering
Hajee Mohammad Danesh Science and Technology University,
Dinajpur-5200



Figure 1: Enter Caption

ABSTRACT

Most universities still run their student clubs on paper forms, spreadsheets, and WhatsApp groups, or else pay for closed, subscription-based engagement suites that cannot be self-hosted or customized. ClubMS was built to close that gap: a self-hosted, white-label club management platform with a Node.js/Express/Prisma REST API on PostgreSQL and a React frontend (Vite, Tailwind, Radix UI). It supports four roles – Platform Admin, Club Admin, Faculty, and Student – under a layered authorization model that checks both global role and per-resource ownership. On top of that sit a full club and membership lifecycle with department-based join rules and a join-agreement flow; multi-club events run through a consent-based co-host invitation system; RSVP tracking; announcements; a venue directory; magic-byte-validated file uploads for logos, banners, and ID cards; and an audit log for accountability. The whole system runs on Docker Compose and can be rebranded for any institution just by changing a few environment variables. Built over about five months, it is backed by 345 automated tests – both unit tests and full HTTP integration tests run against a real, disposable PostgreSQL instance via Testcontainers. This report covers the motivation behind the project, how it compares to existing academic and commercial club-management systems, the requirements gathered from three user groups, the design and implementation, the results, a cost comparison against commercial alternatives, and where the project falls short and could be extended.

Keywords: club management system, role-based access control, white-label web application, React, Node.js.

CONTENTS

ABSTRACT **ii**

1 Introduction **1**

1.1 Background 1
1.2 Problem Statement 2
1.3 Motivation 2
1.4 Objectives 3
1.4.1 General Objective 3
1.4.2 Specific Objectives 3

2 Related Work **4**

2.1 Overview 4
2.2 On-ClubS – UiTM Terengganu (2017) 5
2.3 MERN-Stack Club Management System (2025) 5
2.4 College Club Management System – Framework Proposal (2025) 5
2.5 DYPCETCLUBS (2025) 5
2.6 WeChat Applet Club Management System (2024) 6
2.7 Commercial Campus Engagement Suites 6
2.8 Gap Analysis 7

3 Requirement Specifications **8**

3.1 Stakeholders and User Roles 8
3.2 Stakeholder Consultation and Field Interviews 8
3.2.1 Interview Approach 9
3.2.2 Representative Interview Excerpts 10
3.3 Functional Requirements 12
3.4 Non-Functional Requirements 13
3.5 System Context 14

4 System Development **16**

4.1 Development Methodology 16
4.2 System Architecture 16
4.3 Technology Stack 17
4.4 Database Design 18
4.5 API Design 20

4.6 Authentication and Authorization 20
4.6.1 Authentication Flow 20
4.6.2 Authorization Model 21
4.7 Key Module Implementation 22
4.7.1 Registration and Secure Uploads 22
4.7.2 Club and Membership Lifecycle 22
4.7.3 Multi-Club Event Hosting and Co-Host Invitations 23
4.7.4 Audit Logging 23
4.8 Security Measures 24
4.9 Testing Strategy 24
4.10 Deployment 25

5 Results and Discussion **26**
5.1 Overview 26
5.2 Authentication and Onboarding 26
5.3 Role-Based Dashboards 27
5.4 Club Directory and Detail 28
5.5 Membership Management 30
5.6 Events and RSVP 31
5.7 Platform Administration 32
5.8 Discussion 32

6 Economic Analysis **33**
6.1 Purpose of this Analysis 33
6.2 Development Cost Estimate (Assumption) 33
6.3 Operational Cost Comparison 34
6.4 Benefit Analysis 34

7 Conclusion **35**
7.1 Summary 35
7.2 Limitations 35
7.3 Recommendations and Future Work 36

REFERENCES **37**

LIST OF FIGURES

1 Enter Caption i

1.1 Conceptual positioning of ClubMS between manual coordination methods and commercial SaaS platforms 2

3.1 System context diagram showing ClubMS's principal actors 15

4.1 Detailed system architecture: client tier, containerized frontend/backend/data tiers, and the backend's layered request pipeline . . . 17
4.2 Simplified entity-relationship diagram of the core ClubMS data model . 19
4.3 Login and authenticated request sequence 21
4.4 Role inheritance hierarchy 21
4.5 Co-host invitation lifecycle 23
4.6 Docker Compose deployment topology 25
4.7 Continuous integration pipeline (GitHub Actions) 25

5.1 Registration form with student ID and ID-card upload 26
5.2 Student dashboard: joined clubs and upcoming events 27
5.3 Club admin dashboard: managed clubs, pending requests, and co-host invites 27
5.4 Platform admin dashboard: platform-wide statistics and Manage Clubs access 28
5.5 Club directory: browsable list of all clubs 28
5.6 Club detail page showing roster, announcements, and published rules (member view) 29
5.7 Member management: roles, promote/demote, CSV export 30
5.8 Event listing: all upcoming club events 31
5.9 Event detail page with RSVP controls and attendee count 31
5.10 Admin: Manage Clubs page 32

LIST OF TABLES

2.1 Bibliographic summary of related club/society management systems . . 4
2.2 Feature comparison between related systems and ClubMS 7

3.1 System roles and their primary concerns 8
3.2 Club officers consulted during requirements gathering 9
3.3 Problems reported by club officers and the corresponding ClubMS solution 10
3.4 Functional requirements 12
3.5 Non-functional requirements 14

4.1 Development phases and timeline 16
4.2 Technology stack summary 18
4.3 Core data model entities 19
4.4 Representative API endpoints 20
4.5 Role inheritance map 22
4.6 Implemented security measures 24
4.7 Automated test suite composition 25

6.1 Illustrative development effort and cost estimate (Bangladeshi Taka) . . 33
6.2 Illustrative recurring cost comparison (order-of-magnitude estimates) . . 34

7.1 Limitations mapped to recommended future work 36

CHAPTER 1

INTRODUCTION

1.1 Background

Student clubs and societies are a core part of campus life: they give students a structured way to build skills, form communities, and engage with campus activities outside the classroom. Running them, however, generates a steady stream of administrative work – accepting or rejecting membership applications, tracking who is an active member versus an alumnus, scheduling and publicizing events, recording attendance, and keeping an auditable trail of decisions such as promotions and removals.

In many institutions this work is still done manually. A 2017 case study of the Student Affairs Department at Universiti Teknologi MARA, Terengganu found that club registration and event-proposal processing were handled through paper forms and repeated manual re-entry every semester, motivating the development of a dedicated online club management system [1]. More recent student projects report the same pattern in a different guise: a 2025 case study of club coordination at D. Y. Patil College of Engineering and Technology found that roughly twenty clubs relied on individual WhatsApp groups for communication, leading to fragmented, hard-to-audit information flow [2]. Where institutions do invest in dedicated software, the market is dominated by large, closed, subscription-priced campus-engagement suites such as CampusGroups and Modern Campus Involve (formerly Presence), which bundle orientation management, gamified badging, ticketing, and analytics into a single vendor-hosted product [3]. These products are capable, but they are licensed per institution or per student, cannot be self-hosted, and offer little control over branding, data residency, or the underlying data model.

ClubMS was built to occupy the space between these two extremes: a system that is as structured and auditable as a purpose-built platform, but that a university (or a single department) can deploy on its own infrastructure, brand with its own name, logo, and color, and extend without being tied to a vendor's roadmap or pricing.

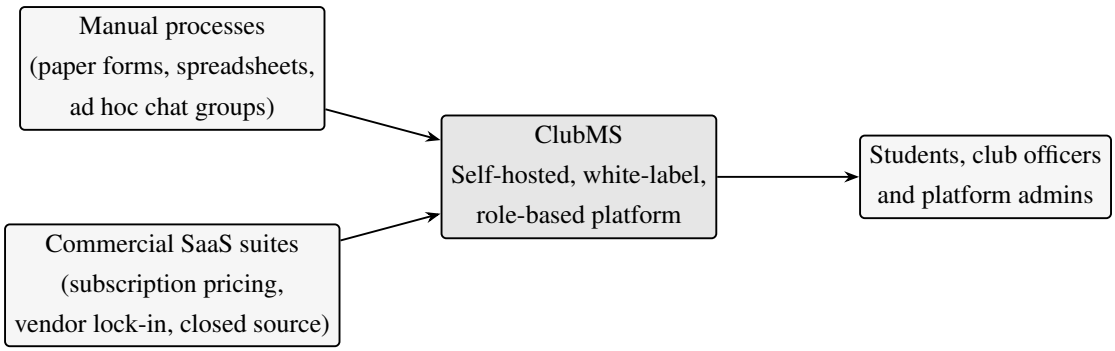


Figure 1.1: Conceptual positioning of ClubMS between manual coordination methods and commercial SaaS platforms

1.2 Problem Statement

Three recurring problems motivate this project:

(a) **Manual, error-prone administration.** Paper-based or spreadsheet-based membership and event tracking does not scale past a handful of clubs, is difficult to audit, and gives no structured way to enforce eligibility rules (for example, restricting membership to particular departments) or to track the lifecycle of a request from submission to approval [1].

(b) **Fragmented, unaccountable communication.** Ad hoc tools such as WhatsApp groups and shared spreadsheets provide no persistent, role-aware record of who requested to join a club, who approved it, who hosted an event, or who attended it [2].

(c) **Vendor lock-in and cost of commercial alternatives.** Existing dedicated platforms are commercially licensed, closed-source, hosted exclusively by the vendor, and priced in a way that is often out of reach for smaller institutions, individual departments, or student-run initiatives, while offering no meaningful way to self-host or white-label the product [3].

There is a clear need for a system that gives institutions the structure and accountability of a dedicated platform while remaining self-hostable, inspectable, and brandable at no licensing cost.

1.3 Motivation

The motivation for this project is both practical and technical. Practically, the author observed the same administrative friction described above in the context of student club coordination, where officers had no systematic way to manage pending join requests,

promote members to officer roles, or coordinate events that involve more than one club. Technically, it was a chance to design and build a production-grade, full-stack system end-to-end rather than a classroom-scale demo. That meant a normalized relational schema that grew through eighteen incremental migrations, an authorization model robust enough to resist both blunt mistakes (a missing role check) and subtler ones (a missing per-resource ownership check), a componentized and accessible frontend, a test suite that hits real HTTP endpoints against a real database instead of mocked collaborators, and a deployment pipeline a real university could actually adopt without much setup.

1.4 Objectives

1.4.1 General Objective

To design, implement, and evaluate a self-hosted, white-label web-based club management platform that digitizes the full lifecycle of university clubs, memberships, and events for three primary user groups (four system roles).

1.4.2 Specific Objectives

1. To design a relational data model and REST API that represent clubs, memberships, events, RSVPs, announcements, and administrative audit trails.
2. To implement a role-based access control (RBAC) system with role inheritance and per-resource ownership checks that scale from a single-club officer to a platform administrator.
3. Implement a complete club and membership lifecycle, including department-based join eligibility rules, a join-agreement flow, and bulk moderation tools for officers.
4. To implement an event management module that supports a single primary host club and a consent-based invitation workflow for co-hosting events across clubs.
5. To implement secure file upload handling (club logos, event banners, and student ID cards) that validates file content rather than trusting client-supplied metadata.
6. To implement a white-labeling mechanism that allows an institution's brand color, logo, and name to be configured without modifying application code.
7. To validate the system with an automated test suite covering both isolated utility functions and full HTTP integration paths against a real database.
8. To package the system for reproducible deployment using Docker and Docker Compose.

CHAPTER 2

RELATED WORK

2.1 Overview

Club and society management is a well-studied problem in student capstone and applied-research literature, and it is also served by a mature commercial market of campus-engagement platforms. This chapter reviews six representative works – five academic projects and one category of commercial products – and positions ClubMS against them. Table 2.1 summarizes each work bibliographically; Table 2.2 compares their features, strengths, and weaknesses relative to the objectives of this project.

Table 2.1: Bibliographic summary of related club/society management systems

#	Author(s) / Year	Title	Platform / Technology
[1]	Syd. Amra (2017)	Online Club Management System for UiTM Terengganu (OnClubS)	PHP/MySQL intranet web system; SDLC with Extreme Programming
[4]	Kumar et al. (2025)	Club Management System: A MERN Stack-Based Web Application for Event Management in Colleges	MongoDB, Express, React, Node.js (MERN)
[5]	Nath et al. (2025)	College Club Management System – Digitizing Student Engagement and Organizational Efficiency	HTML/CSS/JavaScript, Node.js, MySQL, Power BI (framework proposal)
[2]	Ainapure et al. (2025)	DYPCETCLUBS – A Unified Platform for Management of College Clubs	React, TypeScript, Tailwind CSS; Node.js, Express; SQLite with Prisma ORM
[6]	Qi (2024)	Design and Implementation of University Club Management System based on WeChat Applet	WeChat Mini Program (JavaScript) with cloud database
[3]	Modern Campus / CampusGroups (commercial)	Campus engagement platforms (CampusGroups; Modern Campus Involve, formerly Presence)	Proprietary, vendor-hosted SaaS

2.2 On-ClubS – UiTM Terengganu (2017)

On-ClubS automated the Student Affairs Department's paper-based workflow for approving new club proposals and event proposals at a single campus [1]. It was built using a conventional PHP/MySQL stack and evaluated with a usability questionnaire that reported generally positive results. Its scope, however, stops at the proposal-approval stage: it does not model ongoing membership status, does not expose a student-facing join/browse experience, and includes no concept of roles beyond staff versus student. It also targets a single institution's server and has no branding or multi-deployment story.

2.3 MERN-Stack Club Management System (2025)

This paper presents a club and event management system built on the MERN stack (MongoDB, Express, React, Node.js) that automates event scheduling, authentication, participation tracking, and real-time notifications, and explicitly cites role-based access control as a design goal [4]. It is architecturally close to ClubMS's frontend/backend split, but the published work does not describe a membership lifecycle with distinct pending/active/alumni/rejected states, does not address multi-club event collaboration, and does not discuss deployment, white-labeling, or automated testing.

2.4 College Club Management System – Framework Proposal (2025)

This work is a survey- and literature-driven proposal for a "College Club Management System" rather than a fully implemented product; it enumerates the problems of manual club administration and sketches a target feature set based on stakeholder interviews [5]. It is included here because it independently corroborates the problem statement in Chapter 1 – fragmented communication and manual processes – but, being a proposal, it offers no working implementation, authorization model, or evaluation to compare directly against ClubMS.

2.5 DYPCETCLUBS (2025)

DYPCETCLUBS is notable for using a technology stack very close to ClubMS's: React with TypeScript and Tailwind CSS on the frontend, and Node.js/Express with Prisma ORM on the backend [2]. It centralizes event management, notifications, and recruitment for roughly twenty college clubs that previously relied on WhatsApp groups. Its main limitations relative to ClubMS are its use of SQLite (a single-writer, file-based database not well suited to concurrent multi-club production use), the absence of a documented role-inheritance model or per-resource authorization layer, and no mention

of containerized deployment, white-labeling, audit logging, or a consent-based cross-club collaboration workflow.

2.6 WeChat Applet Club Management System (2024)

This system reimplements club management as a WeChat Mini Program backed by a cloud database, aiming to increase interactivity by meeting students inside an app they already use [6]. While effective for institutions where WeChat is the dominant platform, the design is inherently locked to the WeChat ecosystem: it is not accessible as an ordinary web application, has no independent admin dashboard, and cannot be self-hosted or white-labeled outside of WeChat's own infrastructure.

2.7 Commercial Campus Engagement Suites

Platforms such as CampusGroups and Modern Campus Involve are mature, feature-rich SaaS products offering orientation management, virtual involvement fairs, gamified badging, integrated budgeting and payments, and analytics dashboards for large institutions [3]. They are the strongest systems in this comparison in terms of breadth of features and production maturity. Their weaknesses, from the perspective of this project's objectives, are structural rather than functional: they are closed-source, licensed on a recurring commercial basis, hosted exclusively by the vendor (no self-hosting option), and offer only theming-level customization rather than control over the underlying codebase or data model.

2.8 Gap Analysis

Table 2.2: Feature comparison between related systems and ClubMS

System	Strengths	Weaknesses relative to ClubMS
On-ClubS [1]	Automates paper-based proposal workflow; evaluated with real users	No membership lifecycle, no RBAC hierarchy, single-institution, no self-service join flow
MERN CMS [4]	Modern JS stack; RBAC and notifications cited as goals	No documented multi-status membership lifecycle, no multi-club events, no test suite discussed
College CMS proposal [5]	Grounded in stakeholder research; validates the problem statement	Proposal only – no working implementation to compare
DYPCETCLUBS [2]	Near-identical tech stack (React/Tailwind, Node/Express/Prisma)	SQLite (not production-scale), no audit trail, no cross-club collaboration, not white-label
WeChat Applet CMS [6]	High reach within WeChat-native user base	Locked to WeChat platform; not self-hostable or brandable
Commercial suites [3]	Very broad feature set; proven at scale	Closed-source, subscription-priced, vendor-hosted only, no white-label deployment
ClubMS	Self-hosted via Docker Compose; white-labeled at build time; four-role RBAC with inheritance *and* per-resource ownership checks; full PENDING/ACTIVE/ALUMNI/REJECTED membership lifecycle; department-based join rules and join-agreement gating; consent-based cross-club event co-hosting; magic-byte-validated uploads; full audit trail; 345-case automated test suite with a real database	

None of the six systems reviewed here combine self-hosted deployment, white-label branding, a layered RBAC model, consent-based multi-club collaboration, and an integration test suite that runs against a real database. That combination is where ClubMS sits.

CHAPTER 3

REQUIREMENT SPECIFICATIONS

3.1 Stakeholders and User Roles

ClubMS serves three primary human stakeholder groups, mapped onto four system roles (Faculty is a superset of Student and Club Admin permissions, used for faculty advisors who also moderate clubs).

Table 3.1: System roles and their primary concerns

Role	Primary concerns
Student	Discover and join clubs, browse and RSVP to events, manage their own profile, view club announcements.
Club Admin (Officer)	Approve or reject membership requests, promote/demote members, manage events for clubs they officer, post announcements, configure join rules.
Faculty	Advises one or more clubs; inherits Club Admin and Student permissions for the clubs they are associated with.
Platform Admin	University IT / student-affairs staff. Creates and deletes clubs, manages the venue directory, reviews the platform-wide audit log, and has administrative access across all clubs.

3.2 Stakeholder Consultation and Field Interviews

Alongside the literature and product review in Chapter 2, the requirements in this chapter were also shaped directly by informal, in-person interviews with sitting officers of four active student organizations at Hajee Mohammad Danesh Science and Technology University (HSTU). The goal was to ground the feature set in problems that real club officers were currently experiencing, rather than relying solely on assumptions carried over from the reviewed systems. Table 3.2 lists the club officers consulted.

Table 3.2: Club officers consulted during requirements gathering

Club / Organization	Name	Position
CSE Club of HSTU	Eusha Sarwar Utal	Vice President
Programmers Arena	Md. Shohag Showdagor; Kh. Ishtiak Muntasir	Vice President; General Secretary
HSTU Football Club	Nur Ahmed Tonmoy; Samiul Huda Juwel	Assistant Vice President; Registrar
Tangail District Students' Welfare Association-HSTU	Md. Emon Mahabub Emu	Vice President

3.2.1 Interview Approach

Each interview was conducted face-to-face on campus and followed a short, semi-structured format lasting roughly 20–30 minutes: officers were first asked to walk through how their club currently handled membership intake, event coordination, and record keeping, and were then asked what a digital system would need to support for them to actually adopt it over their existing workflow. Notes were taken during each conversation and later cross-checked against the officer's own description of their process. The recurring pain points raised in these conversations, and how each was ultimately addressed in ClubMS, are summarized in Table 3.3.

Table 3.3: Problems reported by club officers and the corresponding ClubMS solution

Reported by	Problem	ClubMS solution
CSE Club of HSTU (Eusha Sarwar Utal)	Membership applications came in through a Google Form and were approved manually from a spreadsheet; there was no way to tell an applicant their request was still pending, and no record of who approved a given member or when.	Structured membership requests with a visible `PENDING` state and officer-only approve/reject actions (FR-7, FR-8), each recorded in the audit log with the approving officer and timestamp (FR-20).
Programmers Arena (Md. Shohag Showdagor, Kh. Ishtiak Muntasir)	Recruitment was open to the whole university even though the club's charter restricts general membership to CSE students, which the officers had no way to enforce except by manually rejecting ineligible applicants after the fact; joint contests with the CSE Club also had no clean way to show both clubs as organizers on one event.	Department-based join-rule configuration that evaluates eligibility automatically at request time (FR-6, FR-7), and a consent-based co-host invitation workflow so a second club can be added as an official event organizer only after its own officer accepts (FR-12, FR-13).
HSTU Football Club (Nur Ahmed Tonmoy, Samiul Huda Juwel)	Attendance for practice sessions and matches was tracked on paper, and building an eligible-player roster for tournament registration meant retyping names from that paper list into a form by hand.	Per-event RSVP tracking with an officer-visible attendee list (FR-15, FR-16) and one-click CSV export of the club roster (FR-10).
Tangail District Students' Welfare Association-HSTU (Md. Emon Mahabub Emu)	Announcements were posted in a WhatsApp group, where older messages quickly scrolled out of view and new members had no way to see the association's history of past notices or club rules.	A persistent, pinnable club announcements panel visible to all members (FR-17) and published club rules text shown to prospective members during the join flow (FR-7).

3.2.2 Representative Interview Excerpts

The exchanges below are paraphrased summaries of the interview conversations, included to illustrate the kind of day-to-day friction that motivated the corresponding features.

Eusha Sarwar Utal, Vice President, CSE Club of HSTU

Q: How does someone currently join the CSE Club?

A: They fill out a Google Form, and one of us copies the responses into a spreadsheet

and messages them once we've decided. It works, but if three of us are handling forms that week, sometimes the same person gets approved twice or nobody follows up at all.

Q: What would make you trust a digital system to replace that?
A: Mainly that it shows the request is still pending instead of the student just messaging us to ask, and that it keeps a record of who approved who – right now if there's a dispute about a membership, nobody can prove what actually happened.

Md. Shohag Showdagor, Vice President & Kh. Ishtiak Muntasir, General Secretary, Programmers Arena

Q: Is membership open to everyone, or restricted?
A (Showdagor): It's open to everyone – any student can sign up regardless of department, no restrictions on our end.

Q: You mentioned joint events with the CSE Club – what's the issue there?
A (Muntasir): When we co-host a programming contest with them, there's no shared listing – we each announce it separately in our own group, so members often don't realize it's the same event, and attendance numbers get split across two announcements.

Nur Ahmed Tonmoy, Assistant Vice President & Samiul Huda Juwel, Registrar, HSTU Football Club

Q: How do you currently track who shows up to practice?
A (Tonmoy): A paper sheet gets passed around at the field. It's fine in the moment, but by the time we need a season attendance summary, half the sheets are missing or smudged.

Q: And for tournament registration?
A (Juwel): I have to go through those same paper sheets and retype an eligible-player list by hand every time. It's the part of being registrar I dread most – it's maybe two hours of just copying names.

Md. Emon Mahabub Emu, Vice President, Tangail District Students' Welfare Association-HSTU

Q: How do you currently reach your members with announcements?
A: A WhatsApp group, like most clubs here. The problem is anything older than a few days is buried under unrelated chat, so a new member has no way to see what the association has announced in the past, or even what our membership rules are, unless someone scrolls back and finds it for them.

Across all four conversations, the same underlying theme recurred: officers were not short on enthusiasm or manual effort, but their tools gave them no durable, shared record of decisions – which is precisely the gap that ClubMS's membership states, audit log, and structured event/announcement model were designed to close.

3.3 Functional Requirements

Table 3.4 lists the functional requirements grouped by module, derived directly from the implemented API surface and frontend pages.

Table 3.4: Functional requirements

ID	Module	Requirement
FR-1	Authentication	Users shall register with an email, password, name, phone number, student ID, and a scanned ID card image; the system shall reject duplicate emails and student IDs.
FR-2	Authentication	Registered users shall authenticate with email and password and receive a signed, time-limited JSON Web Token.
FR-3	Authentication	Users shall be able to view and update their own profile and change their password.
FR-4	Clubs	Platform Admins shall be able to create, update, and delete clubs (name, category, logo).
FR-5	Clubs	Any authenticated user shall be able to browse, search, and paginate the club directory.
FR-6	Clubs	Club officers shall be able to edit a club's description, rules text, and join rules (department allow/block list).
FR-7	Memberships	Students shall be able to request to join a club, subject to department-based join rules and, when configured, an explicit rules-agreement checkbox.
FR-8	Memberships	Club officers shall be able to approve, reject, or bulk-approve/reject pending membership requests.
FR-9	Memberships	Club officers shall be able to promote members to officer, demote officers to member, mark members as alumni, or remove members, subject to self-action guards.
FR-10	Memberships	Officers shall be able to export a club's member list as CSV.
FR-11	Events	Club officers shall be able to create, update, and delete events with a title, description, start/end time, venue, tags, and banner image.
FR-12	Events	Every event shall have exactly one primary hosting club and may have zero or more consenting co-host clubs.
FR-13	Events	An officer of a hosting club shall be able to invite another club to co-host an event; the invited club's officer shall be able to accept, reject, or the inviter may cancel the invitation.

Table 3.4 – continued

ID	Module	Requirement
FR-14	Events	Students shall be able to browse upcoming and past events and filter by club, venue, or "my events".
FR-15	RSVP	Active members of a hosting club shall be able to RSVP to an event as Going, Maybe, or Not Going.
FR-16	RSVP	Officers of a hosting club shall be able to view and export the RSVP list for an event.
FR-17	Announcements	Club officers shall be able to post, pin, and delete announcements visible to a club's members.
FR-18	Venues	Platform Admins shall be able to maintain a directory of reusable, named venues.
FR-19	Uploads	The system shall accept image uploads for club logos, event banners, and ID cards and return a stable URL for the stored file.
FR-20	Audit	The system shall record an audit log entry for administrative actions (club, event, and membership changes) capturing actor, action, target, and metadata.
FR-21	Audit	Platform Admins shall be able to search and paginate the audit log.
FR-22	Dashboard	The system shall present role-appropriate summary statistics (e.g., pending requests for officers, platform totals for admins, upcoming events for students).
FR-23	Departments	The system shall suggest previously used department names to assist officers configuring join rules.

3.4 Non-Functional Requirements

Non-functional requirements were derived from the security, accessibility, and operational goals stated in the project's design documentation, and from established best-practice references for web application security [10] and accessibility [9].

Table 3.5: Non-functional requirements

Category	Requirement
Security	Passwords shall be hashed (never stored in plain text); all state-changing endpoints shall require a valid JWT; authorization shall be enforced both by global role and by per-resource ownership; uploaded files shall be validated by content (magic bytes), not by client-supplied extension or MIME type [10].
Performance	List endpoints shall be paginated (default 20–50 items, maximum 100); frequently filtered/sorted columns shall be indexed; the frontend shall code-split routes so users only download the JavaScript needed for the page they are viewing.
Availability & Reliability	The API shall retry idempotent read operations transparently on transient database connection errors, but shall never automatically retry non-idempotent write operations.
Usability & Accessibility	The interface shall meet WCAG 2.1 AA expectations: sufficient color contrast, full keyboard navigation, visible focus states, screen-reader-friendly semantics, and respect for the user's reduced-motion preference [9].
Maintainability	The backend shall be organized as isolated feature modules (routes/controller/service/schema) with input validation centralized through a single schema-validation middleware.
Portability & White-labeling	The system shall be deployable via a single Docker Compose command; the frontend's brand color, logo, and institution name shall be configurable through build-time environment variables without code changes.
Testability	The system shall be covered by an automated test suite exercising both isolated utility logic and full HTTP request/response behavior against a real database instance.
Auditability	Every privileged administrative action shall leave a permanent, queryable audit record.

3.5 System Context

Figure 3.1 shows the system context: three actor classes interact with the ClubMS platform over HTTPS, and a fourth "deployment" concern – the institution's IT staff – configures the platform once at deployment time via environment variables rather than through the application itself.

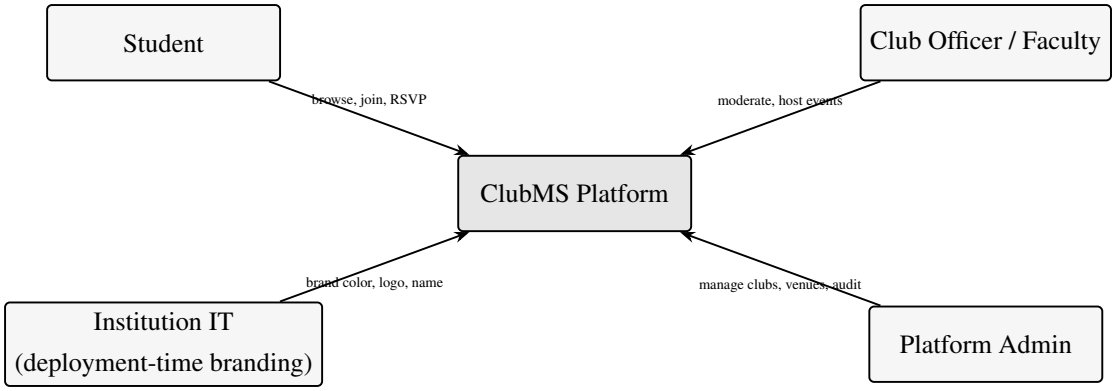


Figure 3.1: System context diagram showing ClubMS's principal actors

CHAPTER 4

SYSTEM DEVELOPMENT

4.1 Development Methodology

ClubMS was developed iteratively rather than through a single up-front design phase, following an approach close to Agile feature-slicing: each iteration delivered a vertically complete slice of functionality (database migration, service logic, route, and corresponding UI) before moving to the next. Table 4.1 summarizes the major development phases as recorded in the project's version-control history, spanning approximately five months.

Table 4.1: Development phases and timeline

Period	Milestone
Apr. 2026	Project foundation: Express/Prisma backend skeleton, JWT authentication, initial React shell, routing, and layout.
Apr.–May 2026	Club directory and CRUD, membership lifecycle with role hierarchy, events CRUD with calendar view, seed/deseed tooling, security hardening pass.
May 2026	RSVP, dashboard analytics, accessibility pass, skeleton loading states, file upload with dark-mode support, code splitting, first audit-logging pass, initial automated test suite, CI pipeline.
Jun. 2026	Self-action guards on privileged operations; multi-host events with a dedicated co-host feature.
Jul. 2026	Upload security hardening (magic-byte validation, CSP-locked static serving) and read-retry logic; managed venue directory and admin UI.
Aug. 2026	Consent-based cross-club co-host invitations; multipart registration with student ID and ID-card upload; department-based club join rules; club rules and join-agreement flow; membership position field; final UI and accessibility polish.

4.2 System Architecture

ClubMS follows a three-tier client-server architecture: a React single-page application (SPA) communicates with a stateless Express REST API over JSON, and the API is the sole component that talks to the PostgreSQL database, through Prisma ORM. All three components (database, backend, frontend) are packaged as separate Docker containers orchestrated by Docker Compose. Figure 4.1 shows the full picture: the client tier,

the two containerized service tiers, and the layered request pipeline inside the backend container – security middleware (Helmet, CORS, rate limiting), JWT authentication, role-based access control, Zod schema validation, the controller, the service layer where business rules and transactions live, and finally Prisma ORM for database access. The service layer also writes directly to the `/uploads` volume for validated file storage, bypassing the ORM entirely since uploaded files are not relational data.

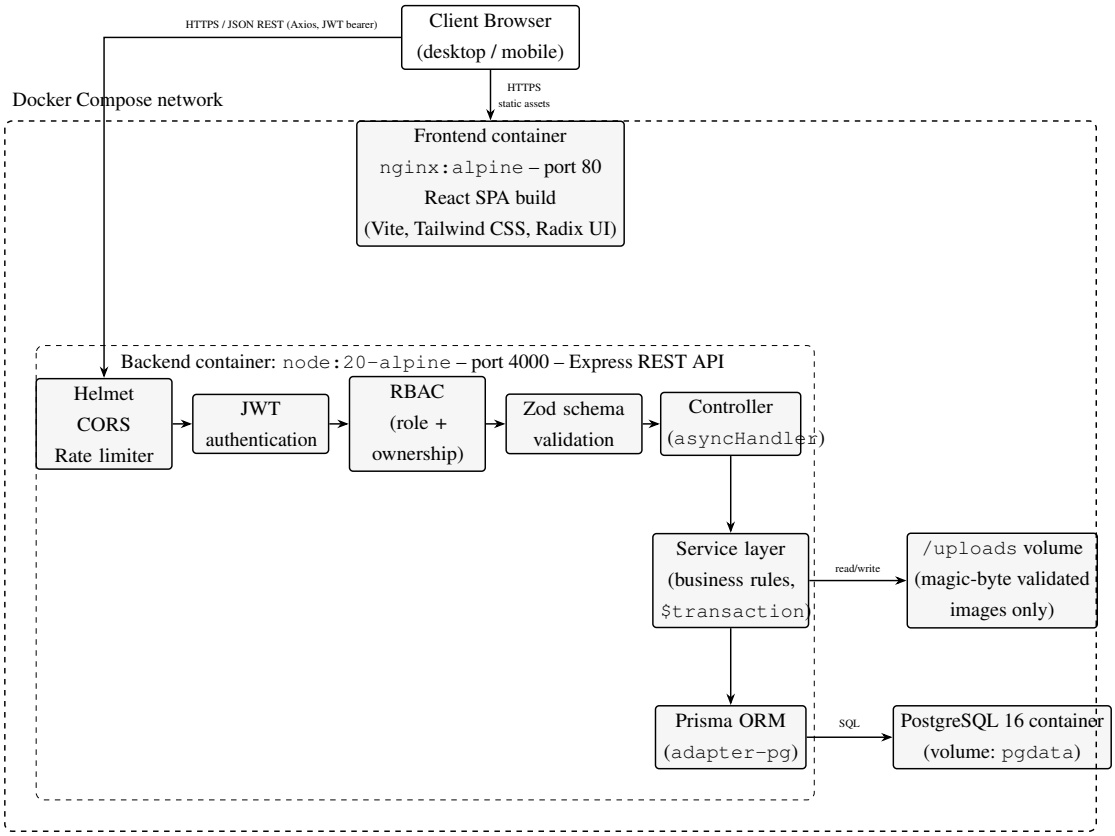


Figure 4.1: Detailed system architecture: client tier, containerized frontend/backend/data tiers, and the backend's layered request pipeline

4.3 Technology Stack

Table 4.2 lists the primary technology choices per architectural layer.

Table 4.2: Technology stack summary

Layer	Technologies
Frontend	React 19 [11], Vite 6, React Router 7, Tailwind CSS v4 [18], Radix UI primitives [19], class-variance-authority, Axios, Sonner (toasts), Lucide icons
Backend	Node.js, Express 4 [12], Zod schema validation [20], JSON Web Tokens [16], bcryptjs [17], Multer, Helmet, express-rate-limit, Pino structured logging, Swagger/OpenAPI documentation
Data layer	PostgreSQL 16 [14], Prisma ORM 7 with the `@prisma/adapter-pg` driver adapter [13]
Testing	Jest, Supertest, Testcontainers (ephemeral PostgreSQL per test run) [21]
DevOps	Docker and Docker Compose [15], GitHub Actions CI

4.4 Database Design

The relational schema evolved through eighteen incremental Prisma migrations, from the initial schema in April 2026 to the addition of membership positions in August 2026. The core entities are `User`, `Club`, `ClubMembership`, `Event`, `EventHost`, `EventHostInvite`, `Venue`, `EventRsvp`, `Announcement`, and `AuditLog`. Figure 4.2 shows a simplified entity-relationship diagram of the core scheduling and membership entities, drawn in Chen notation: rectangles denote entity sets, diamonds denote named relationship sets, and ovals denote attributes (underlined = primary key, `...Id` = foreign key), with each edge annotated with its cardinality. The four join tables (`ClubMembership`, `EventHost`, `EventHostInvite`, `EventRsvp`) appear as associative entities at the many side of their access paths, exactly as they are materialized in the relational schema. `Announcement` and `AuditLog` (each a straightforward one-to-many child of `Club` and/or `User`) are omitted from the diagram for clarity and are described in Table 4.3 instead.

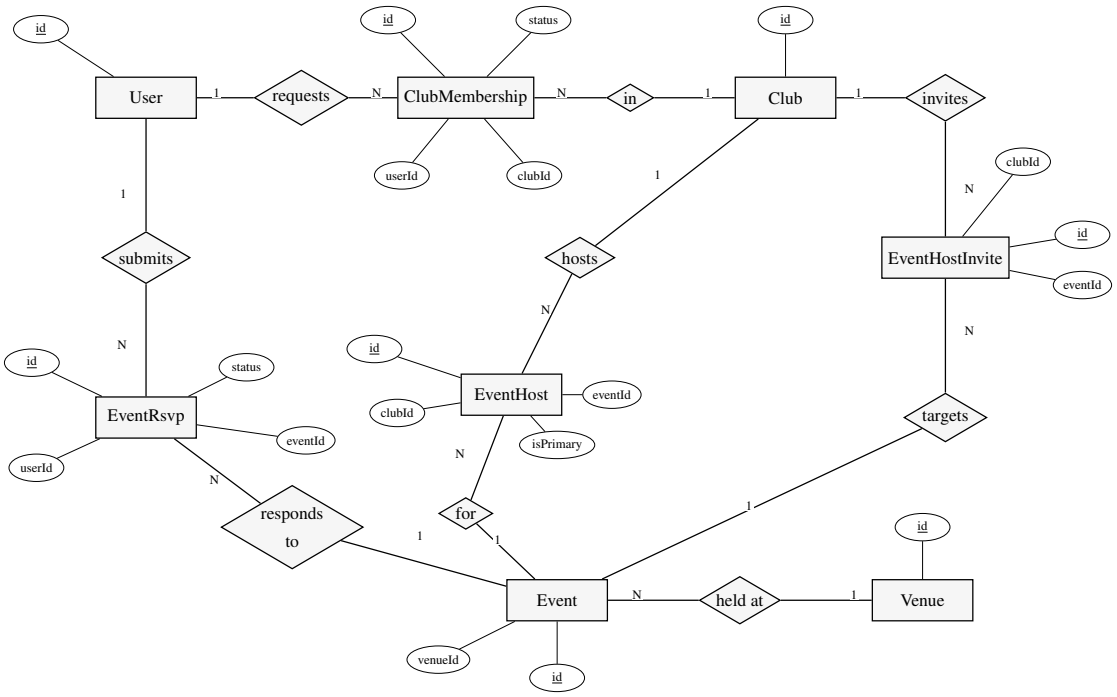


Figure 4.2: Simplified entity-relationship diagram of the core ClubMS data model

Table 4.3: Core data model entities

Entity	Description
User	Account record: email, hashed password, role (Student/Club Admin/Faculty/Platform Admin), name, phone, student ID, ID-card image URL, and optional department/home-district.
Club	Name, category, logo, description, rules text, and department-based join-rule configuration (mode: all/allow-list/block-list).
ClubMembership	Join record between a User and a Club: status (Pending/Active/Alumni/Rejected), member role (Member/Officer), and an optional position title. Unique per (user, club).
Event	Title, description, start/end time, venue snapshot, optional foreign key to a managed Venue, banner image, and tags.
EventHost	Join table recording which clubs host an event; exactly one row per event has `isPrimary = true`.
EventHostInvite	Pending, not-yet-accepted co-host invitation between an Event and a Club; deleted on accept (converted into an `EventHost` row) or on reject/cancel.
Venue	Reusable, named location that can be attached to events for filtering.
EventRsvp	A user's RSVP status (Going/Maybe/Not Going) for an event; unique per (event, user).
Announcement	Club-scoped message, optionally pinned, authored by an officer.
AuditLog	Immutable record of an administrative action: actor, action name, target type/ID, JSON metadata, and IP address.

Two design decisions in the schema are worth highlighting. First, `Event` has no direct `clubId` column; instead, the `EventHost` join table is the single source of truth for which clubs host an event, which is what makes multi-club co-hosting possible without duplicating events. Second, `EventHostInvite` deliberately stores only *pending* invitations rather than a status column on `EventHost` itself, so that authorization logic which queries `EventHost` (for example, "is this user an officer of a hosting club") never has to filter out unaccepted invitations – an un-accepted invite simply does not exist as a host relationship yet.

4.5 API Design

The API follows conventional REST semantics under the `/api` prefix, documented interactively through Swagger/OpenAPI at `/api/docs`. List endpoints return a consistent paginated envelope, `{ items, total, page, limit, totalPages }`, and accept `page`, `limit`, and (where applicable) `search` query parameters. Table 4.4 lists a representative subset of the full endpoint surface.

Table 4.4: Representative API endpoints

Method	Path	Purpose
POST	/api/auth/register	Multipart registration with ID-card upload
POST	/api/auth/login	Authenticate and receive a JWT
GET	/api/clubs	Paginated, searchable club directory
POST	/api/clubs/:clubId/join	Request membership in a club
PATCH	/api/clubs/:clubId/members/:userId/promote	Promote a member to officer
POST	/api/events/:id/cohost-invites	Invite another club to co-host an event
POST	/api/cohost-invites/:id/accept	Accept an incoming co-host invitation
PUT	/api/events/:id/rsvp	Set the caller's RSVP status for an event
GET	/api/events/:id/rsvps	List (officer-only) RSVPs for an event
GET	/api/audit-logs	Paginated, filterable audit trail (Platform Admin only)
POST	/api/upload	Validate and store an image, returning its URL

4.6 Authentication and Authorization

4.6.1 Authentication Flow

Authentication uses stateless JSON Web Tokens [16]. On login, the server verifies the submitted password against a bcrypt hash [17] and, on success, signs a 24-hour token containing the user's ID, email, and role. The client attaches this token as a Bearer

credential on every subsequent request; middleware verifies the signature and expiry before attaching the decoded identity to the request object. Figure 4.3 illustrates this flow.

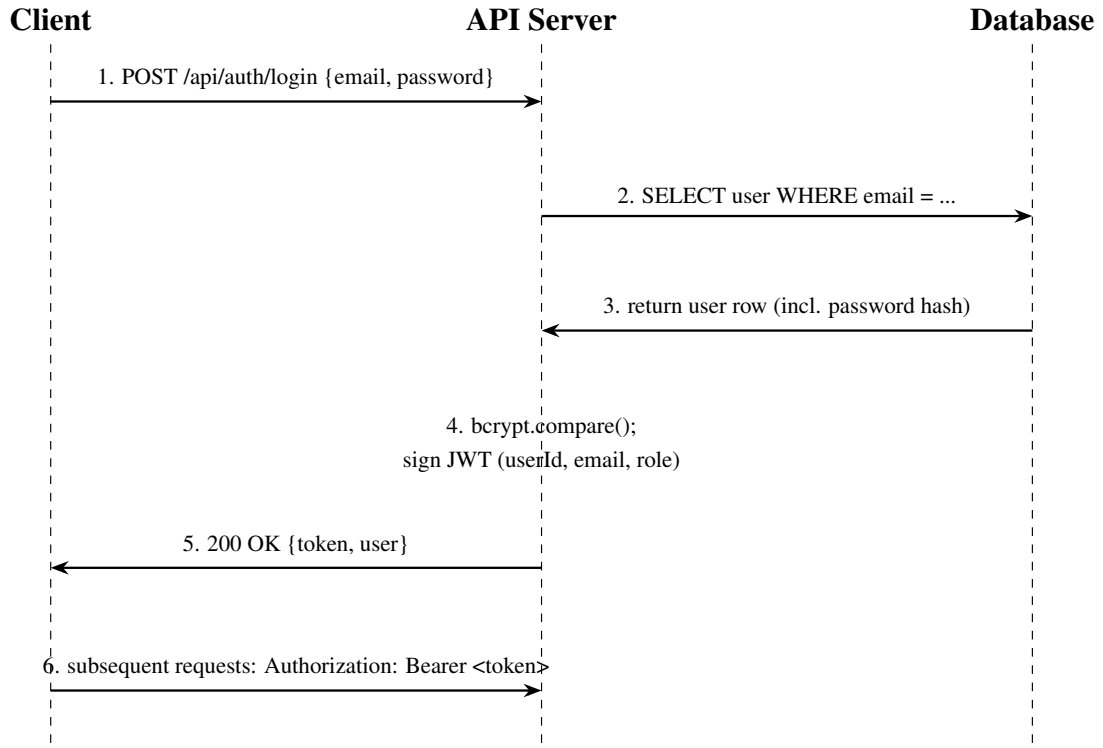


Figure 4.3: Login and authenticated request sequence

4.6.2 Authorization Model

Authorization is enforced in two layers. The first is a global role check, `requireRole(...)`, which consults a static inheritance map so that a Platform Admin automatically satisfies any Club Admin check, and a Faculty user automatically satisfies both Club Admin and Student checks (Figure 4.4, Table 4.5). The second is a per-resource ownership check, `requireClubAdmin`, which verifies that the caller is actually an officer of the specific club referenced by the request – without this second layer, any Club Admin (of any club) could modify any other club's data.

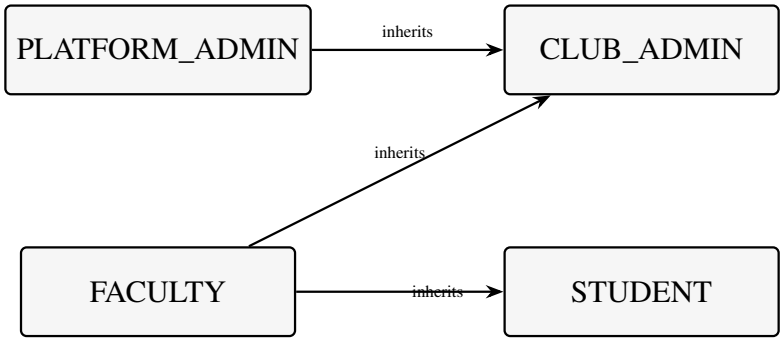


Figure 4.4: Role inheritance hierarchy

Table 4.5: Role inheritance map

Assigned role	Satisfies role checks for
PLATFORM_ADMIN	PLATFORM_ADMIN, CLUB_ADMIN
FACULTY	FACULTY, CLUB_ADMIN, STUDENT
CLUB_ADMIN	CLUB_ADMIN
STUDENT	STUDENT

This design underpins several theoretical foundations of role-based access control on the web [7,8]: roles decouple permission assignment from individual identities, while the additional per-resource check implements the principle of least privilege at the level of an individual club rather than the platform as a whole.

4.7 Key Module Implementation

4.7.1 Registration and Secure Uploads

Registration is a multipart form submission combining text fields (name, email, password, phone, student ID) with an ID-card image file. The uploaded file is never trusted by its declared MIME type or filename extension; instead, its first bytes are inspected to determine the true image format, and the file is persisted under a randomly generated name with an extension derived from that detection. The same validated-upload helper is reused for club logos, event banners, and student ID cards, and the entire `/uploads` directory is served with a strict `Content-Security-Policy: default-src 'none'` header so that even a successfully uploaded file can never execute script in the browser.

4.7.2 Club and Membership Lifecycle

A club's join behavior is governed by a `joinRuleMode` of `ALL`, `ALLOW`, or `BLOCK` combined with a list of departments; `requestJoin` evaluates this rule, and – if the club has published rules text – requires the requester to explicitly agree before the request is created. Membership then moves through `PENDING` → `ACTIVE`/`REJECTED`, with a rejected user able to re-apply (which resets the same row to `PENDING` rather than creating a duplicate). Officers can additionally promote a member to `OFFICER` (which also elevates the user's global role to `CLUB_ADMIN`), demote an officer back to member, mark a member as `ALUMNI`, or remove a membership outright, with guards that prevent a user from demoting or removing themselves and that block removing the club's only remaining officer.

4.7.3 Multi-Club Event Hosting and Co-Host Invitations

Every event has exactly one primary hosting club and may accumulate co-hosts through a consent-based invitation flow, shown in Figure 4.5. An officer of any existing host club may invite another club; an officer of the invited club must explicitly accept before an `EventHost` row is created, or may reject it, and the inviter may cancel an outstanding invitation. This mirrors the same pending-then-confirmed pattern already used for membership requests, and keeps unaccepted invitations completely invisible to authorization checks that determine who may edit or RSVP to an event.

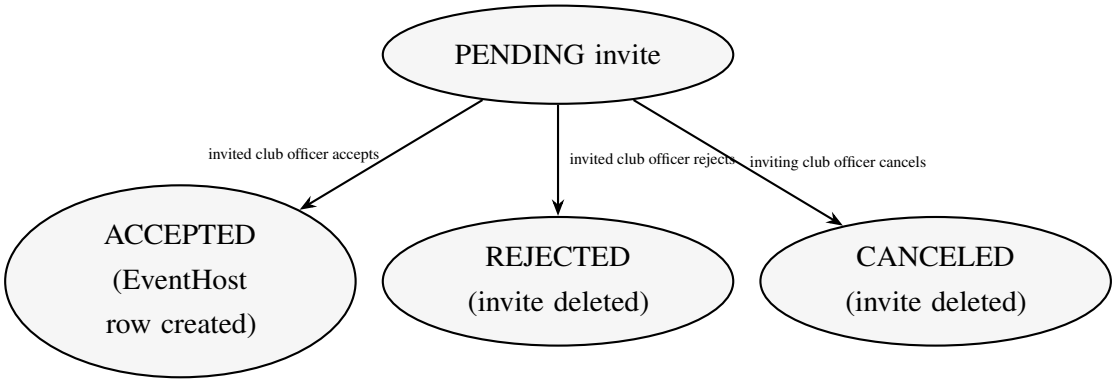


Figure 4.5: Co-host invitation lifecycle

4.7.4 Audit Logging

Administrative actions across the clubs, events, and memberships modules are recorded through an Express middleware that patches the response lifecycle to fire an asynchronous, non-blocking write to the `AuditLog` table after a successful response is sent. This keeps the client-perceived latency unaffected by logging while still producing a durable trail of actor, action, target, and metadata for every privileged mutation, queryable by Platform Admins.

4.8 Security Measures

Table 4.6: Implemented security measures

Concern	Mitigation
Transport / headers	Helmet-applied security headers on every response; strict CSP on the uploads directory
Password storage	bcrypt hashing with a per-password salt [17]
Session integrity	Signed, expiring JWTs; secret required at startup with no insecure default
Cross-origin requests	CORS restricted to the configured frontend origin
Brute force / abuse	Rate limiting: 20 requests/15 min on auth endpoints, 100 requests/min generally
Injection	All queries parameterized through Prisma; no raw string-concatenated SQL
File upload	Magic-byte content validation; extension derived from detected type, never from client input; 5 MB size cap; memory-buffered scan before any disk write
Payload size	JSON body size capped at 10 KB
Privilege escalation	Two-layer authorization (global role + per-resource ownership); self-action guards on demote/remove
Accountability	Immutable audit log of privileged actions [10]

4.9 Testing Strategy

The test suite is split into two layers. Pure unit tests exercise dependency-free utility functions (input validation helpers, the database-retry policy, and the upload magic-byte sniffer) with no I/O. HTTP integration tests build the real Express application (bypassing only `listen()`) and issue real requests through Supertest against a disposable PostgreSQL 16 instance provisioned per test run by Testcontainers [21], exercising the full middleware chain – rate limiting, JWT verification, RBAC, Zod validation, controllers, services, and the audit-log middleware – exactly as it runs in production. The database is reset between tests by truncating all tables, and factory functions build exactly the rows each test needs rather than relying on a shared global seed.

Table 4.7: Automated test suite composition

Layer	Coverage
Unit tests	Validation helpers, database retry policy, upload content-type detection, static file headers
Integration tests	Authentication, clubs, memberships, events, RSVP, announcements, co-host invitations, venues, departments, audit log, uploads, dashboard statistics, and platform health
Total test cases	345 automated test cases
Continuous Integration	GitHub Actions runs the full suite on every push

4.10 Deployment

The system is packaged as three Docker images orchestrated by a single `docker-compose.yml`: a PostgreSQL 16 database with a health check and a named volume for durable storage; a Node.js backend image that runs pending Prisma migrations before starting the API server; and a multi-stage frontend image that builds the React application with the institution's branding baked in as build arguments, then serves the static output through Nginx. Figure 4.6 shows the container topology, and Figure 4.7 shows the continuous-integration pipeline that validates every change before it reaches deployment.

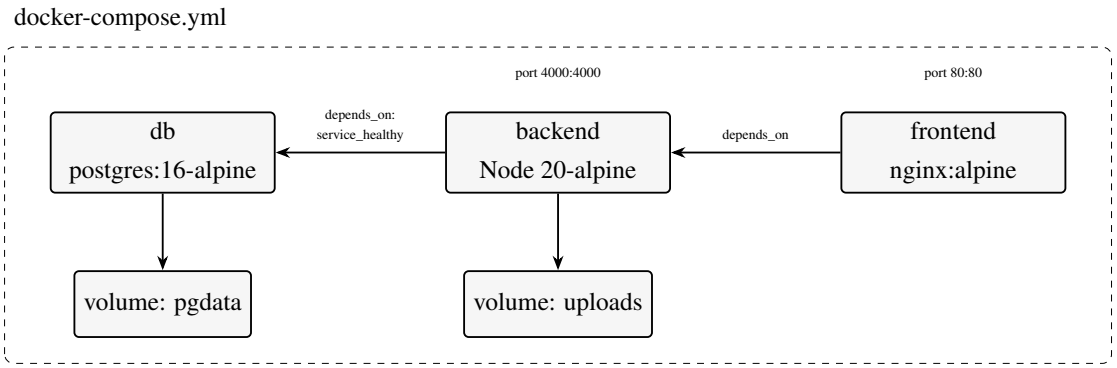


Figure 4.6: Docker Compose deployment topology

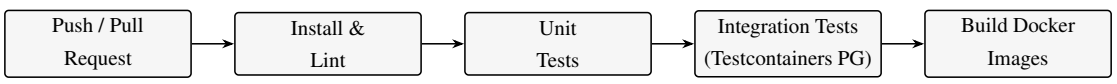


Figure 4.7: Continuous integration pipeline (GitHub Actions)

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Overview

This chapter presents the delivered application organized by functional area, with a placeholder for a representative screenshot at each point. Figures should be captured from a running deployment (seeded with the sample data described in the project's seeding guide) and inserted in place of each boxed placeholder.

5.2 Authentication and Onboarding

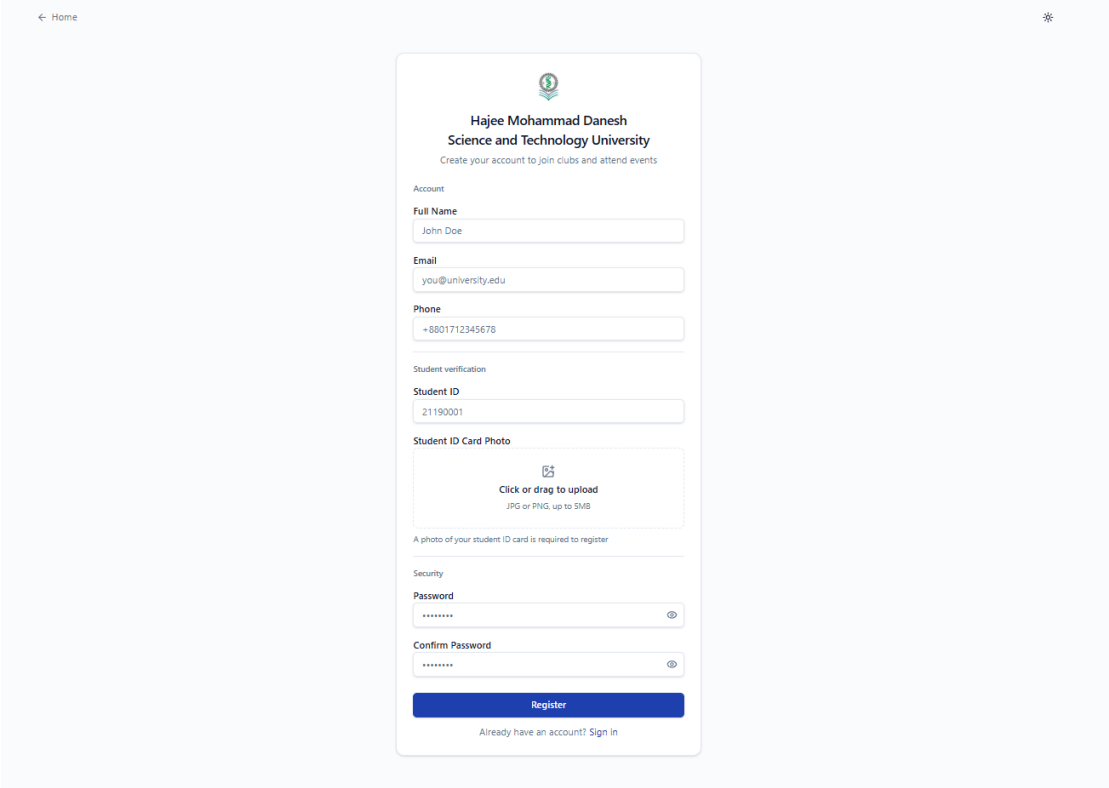


Figure 5.1: Registration form with student ID and ID-card upload

Figure 5.1 shows the registration form. New users register with their institutional details and a scanned ID card; the system rejects duplicate emails and student IDs at submission time and shows inline validation errors sourced directly from the backend's Zod schemas.

5.3 Role-Based Dashboards

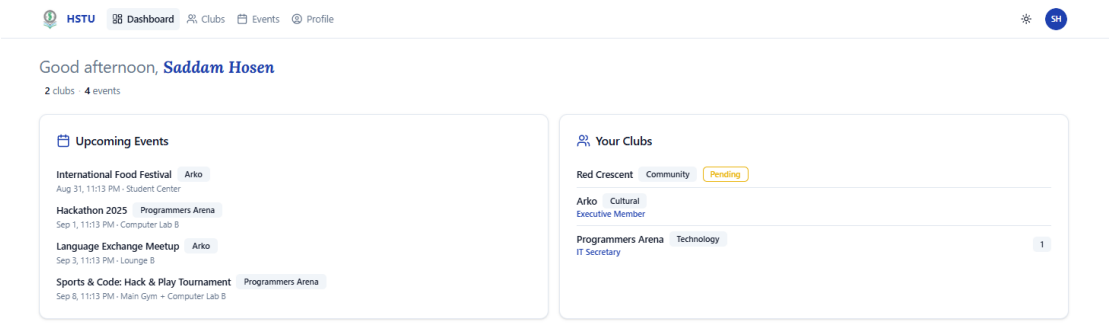


Figure 5.2: Student dashboard: joined clubs and upcoming events

Figure 5.2 shows the student dashboard: a statistics strip of the student's joined clubs and upcoming events, giving a one-glance view of their own activity.

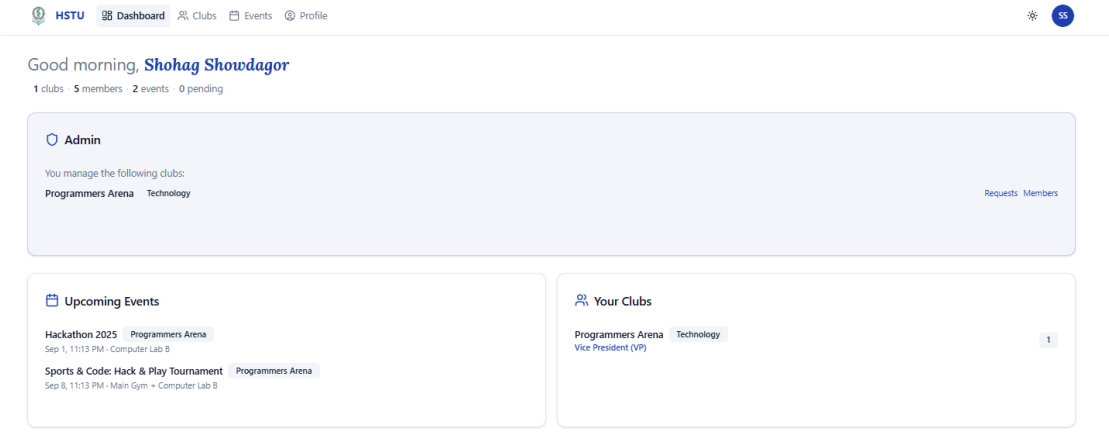


Figure 5.3: Club admin dashboard: managed clubs, pending requests, and co-host invites

Figure 5.3 shows the club admin dashboard: the clubs the officer manages, with quick links into pending requests and the member roster, plus incoming co-host invitations awaiting a response.

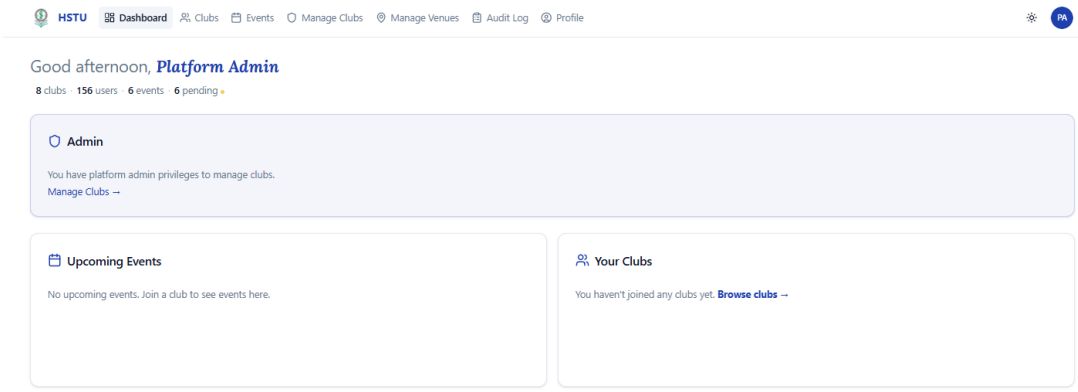


Figure 5.4: Platform admin dashboard: platform-wide statistics and Manage Clubs access

Figure 5.4 shows the platform admin dashboard: platform-wide totals (clubs, users, events, pending memberships) with a direct link into the Manage Clubs page. All three dashboards render a role-specific statistics strip and hide sections entirely rather than disabling them for viewers who are not entitled to see that data.

5.4 Club Directory and Detail

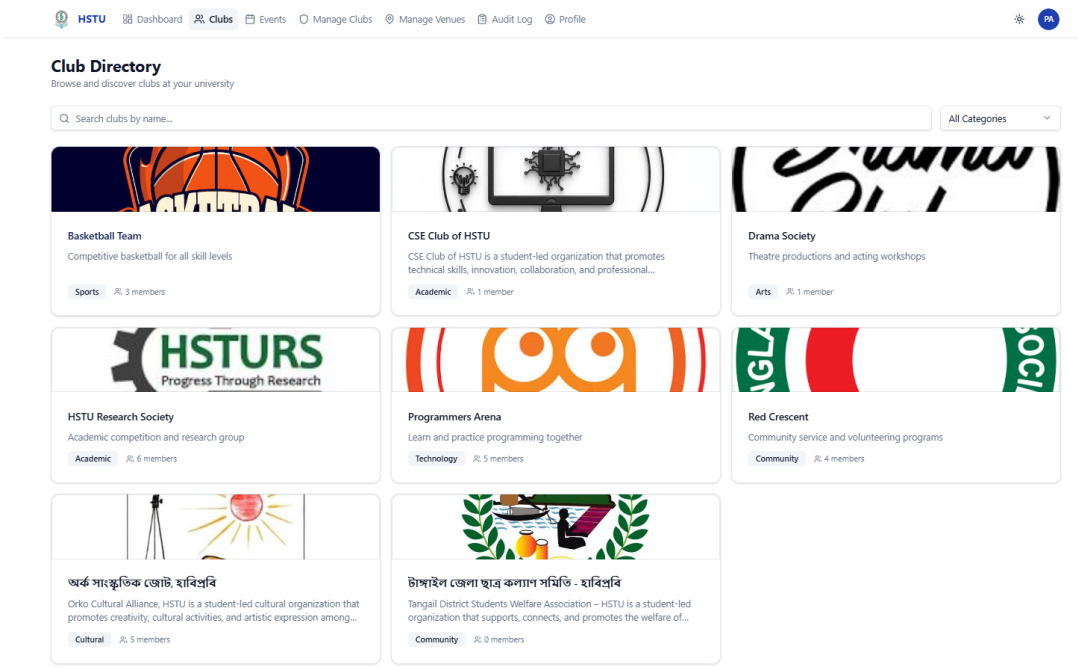


Figure 5.5: Club directory: browsable list of all clubs

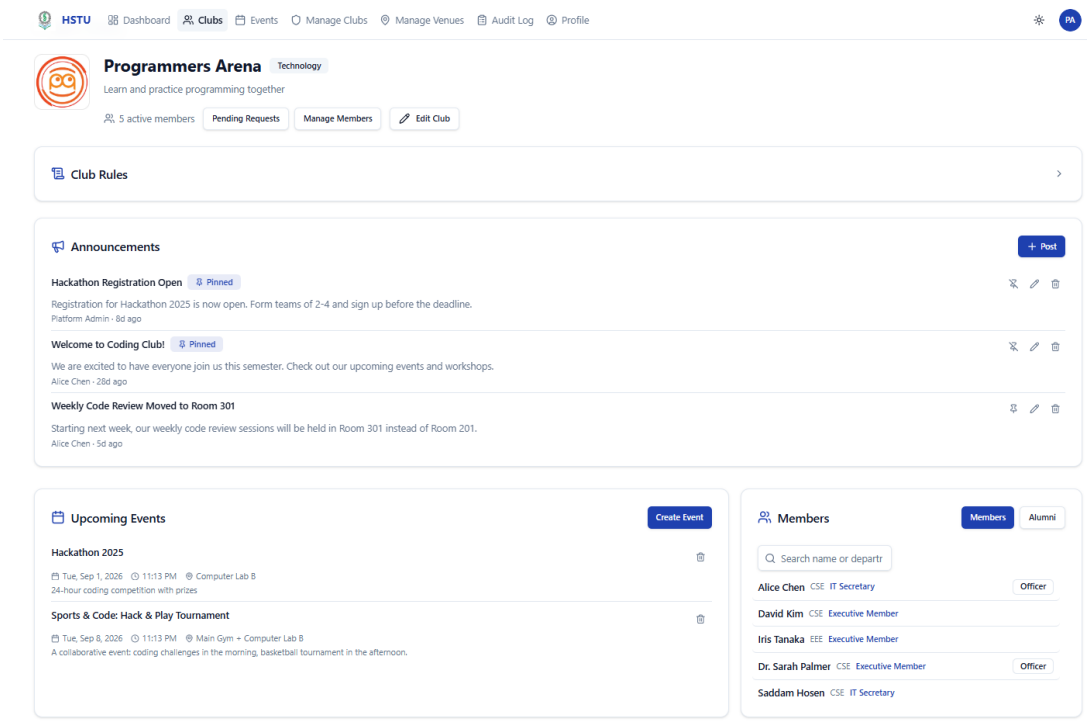


Figure 5.6: Club detail page showing roster, announcements, and published rules (member view)

Figure 5.5 shows the club directory, which lists every club with its category and member count and lets a visitor search and filter before opening a club's detail page. Figure 5.6 shows the resulting detail page; it reveals the member roster, contact details, and announcements only to active members and officers, while non-members see the club's public description and published rules without any unauthorized data ever leaving the server.

5.5 Membership Management

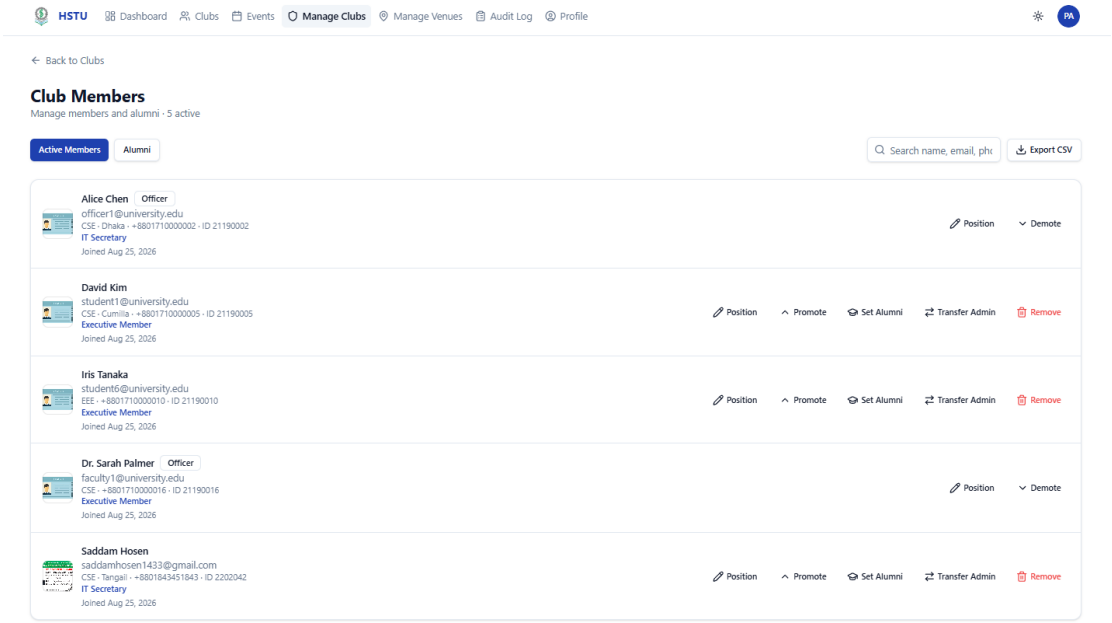


Figure 5.7: Member management: roles, promote/demote, CSV export

Figure 5.7 shows the member management page. Officers approve or reject pending applicants in bulk, promote members to Officer or demote them back to Member, and export the full roster as a CSV file for offline record-keeping.

5.6 Events and RSVP

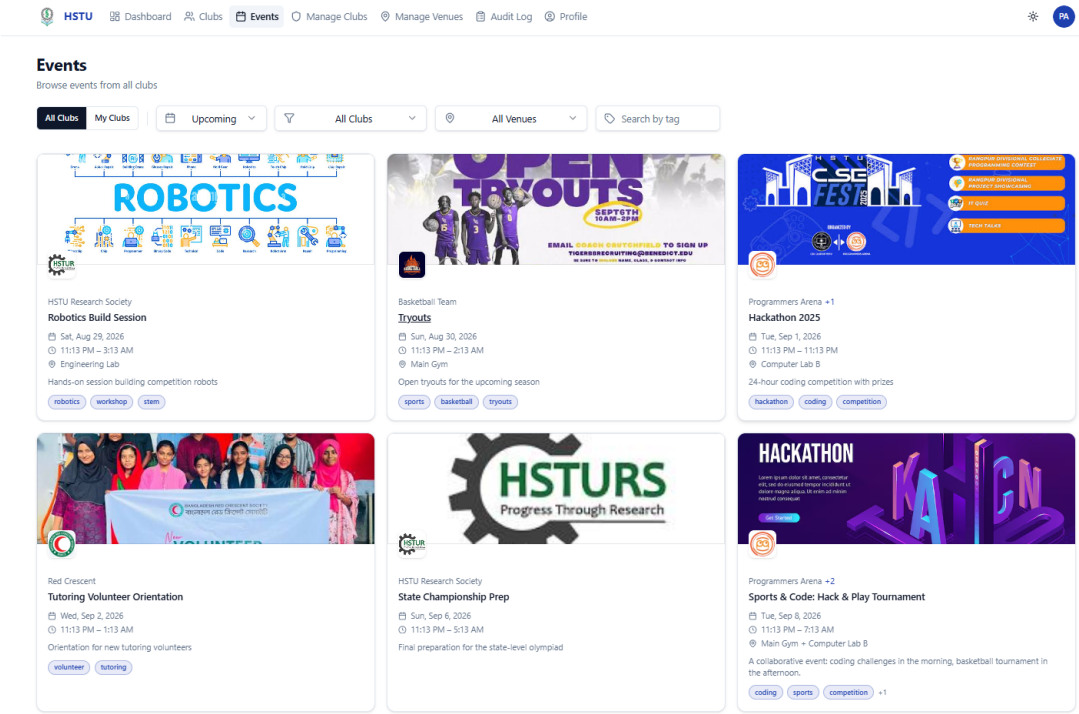


Figure 5.8: Event listing: all upcoming club events

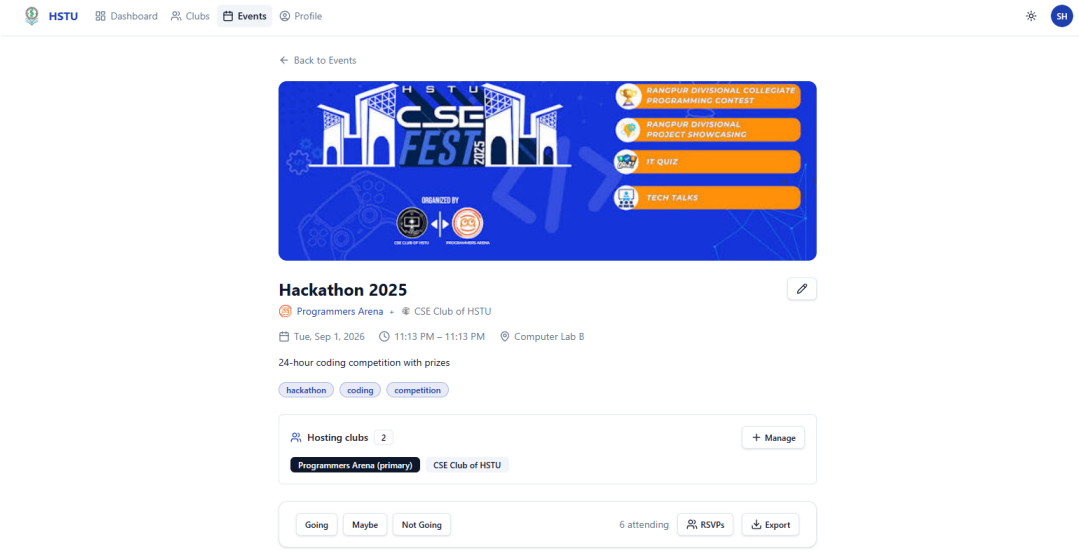


Figure 5.9: Event detail page with RSVP controls and attendee count

Students browse all upcoming events across clubs (Figure 5.8) and then RSVP as Going, Maybe, or Not Going on the event detail page (Figure 5.9); the attendee count updates immediately, and officers of any hosting club can additionally view and export the full RSVP list.

5.7 Platform Administration

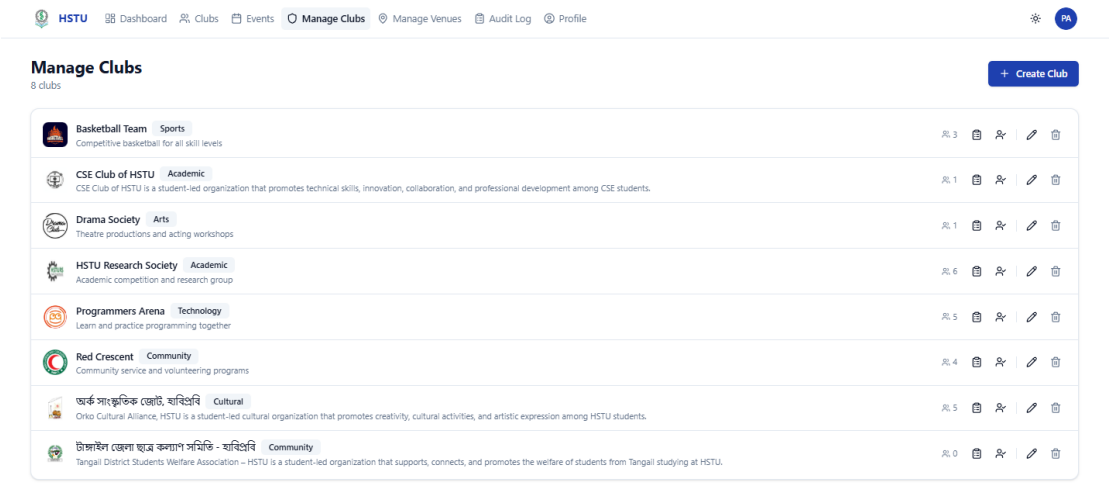


Figure 5.10: Admin: Manage Clubs page

Figure 5.10 shows the Manage Clubs page. Platform admins create and edit clubs, assign each one a name, category, logo, and brand color, and the same page surfaces every club's join-rule configuration for quick review.

5.8 Discussion

Measured against the objectives set out in Section 1.4, the delivered system covers all eight of them. The schema, evolved through eighteen migrations, models every entity described in Chapter 4, and the RBAC layer combines role inheritance with per-resource ownership checks. The club and membership lifecycle – department-based eligibility, join agreements, and all – is implemented and tested end to end, and multi-club events work through the consent-based invitation model described in Section 4.7.3. Uploads are validated by content rather than trusted metadata, and white-labeling comes down to three build-time environment variables. The whole system is backed by 345 automated tests spanning unit and full-stack integration levels, and it ships as a single Docker Compose file.

Compared with the systems reviewed in Chapter 2, ClubMS's advantage is structural rather than a matter of feature count: none of the academic projects documented a per-resource authorization layer, a consent-based cross-club workflow, or an integration suite tested against a real database, and none of the commercial suites can be self-hosted or white-labeled at all.

CHAPTER 6

ECONOMIC ANALYSIS

6.1 Purpose of this Analysis

Because ClubMS is a self-hosted, open codebase produced as an academic project rather than a commercial product with an existing pricing history, the figures in this chapter are illustrative estimates intended to demonstrate the *shape* of the cost-benefit tradeoff – self-hosted, one-time development effort versus recurring commercial licensing – rather than audited financial figures. They should be recalibrated with real quotations before being used for an actual procurement decision.

6.2 Development Cost Estimate (Assumption)

Table 6.1 breaks the observed development effort down by phase (derived from the project's iteration history in Table 4.1) and converts it to an illustrative cost in BDT, using an assumed blended day rate of BDT 1,500 – representative of a junior-to-mid-level software developer's rate in the Bangladeshi IT job market.

Table 6.1: Illustrative development effort and cost estimate (Bangladeshi Taka)

Phase	Effort (person-days)	Rate (BDT/day, assumed)	Cost (BDT)
Foundation, auth, and core CRUD	12	1,500	18,000
Membership and club lifecycle	8	1,500	12,000
Events, RSVP, and co-hosting	10	1,500	15,000
Admin tools, audit log, venues	7	1,500	10,500
UI/UX, accessibility, theming	8	1,500	12,000
Security hardening	5	1,500	7,500
Automated testing and CI/CD	6	1,500	9,000
Deployment packaging and documentation	4	1,500	6,000
Total	**60**		**90,000**

6.3 Operational Cost Comparison

Table 6.2 compares the recurring cost profile of self-hosting ClubMS against the general pricing pattern of commercial campus-engagement suites, which are typically licensed on a per-institution or per-student annual basis rather than a flat hosting fee [3].

Table 6.2: Illustrative recurring cost comparison (order-of-magnitude estimates)

Cost driver	Self-hosted ClubMS	Commercial SaaS suite
Hosting / infrastructure	Small VPS or on-prem server, approx. USD 10–30/month	Included in subscription
Software licensing	None (source available; MIT-style internal use)	Annual per-institution or per-student license fee
Customization / branding	Free – environment variables and, if needed, direct code changes	Limited to vendor-exposed theming options
Data ownership	Full control; data never leaves institutional infrastructure	Hosted by vendor; export/portability depends on contract
Vendor lock-in risk	Low	High
Feature breadth (out of the box)	Core club/event/membership workflow	Broad (ticketing, gamification, budgeting, analytics)

6.4 Benefit Analysis

The primary quantifiable benefit of digitizing club administration is officer time saved: bulk approve/reject of pending membership requests and CSV export of rosters and RSVP lists both replace what would otherwise be manual, per-record work. Qualitatively, the platform also reduces the error rate inherent in spreadsheet-based tracking (duplicate join requests are prevented at the database level by a unique constraint), and the audit log gives student-affairs staff a way to answer "who approved this, and when" questions that ad hoc tools cannot answer at all.

CHAPTER 7

CONCLUSION

7.1 Summary

This project set out to design and build a self-hosted, white-label club management platform that could replace the manual, fragmented processes many university clubs still rely on, without pushing the institution toward a closed, subscription-priced SaaS product. The delivered system, ClubMS, covers the full club and membership lifecycle with department-based eligibility rules, a multi-club event model with consent-based co-host invitations, RSVP tracking, announcements, and a managed venue directory. Underneath that sits a four-role RBAC system combining global and per-resource authorization, secure content-validated file uploads, a full administrative audit trail, and white-label theming configurable at deployment time. All of it is validated by 345 automated tests – including HTTP-level integration tests run against a real, disposable PostgreSQL database – and packaged for one-command deployment with Docker Compose. As Chapter 2 showed, none of the five academic/student projects or the commercial suites reviewed combine all of these properties, which is what this project set out to contribute.

7.2 Limitations

- **Single-tenant deployment.** One running instance of ClubMS serves exactly one institution; there is no built-in multi-tenant mode for a single deployment to serve several universities with isolated data.
- **No native mobile application.** The frontend is a responsive web SPA; there is no dedicated iOS/Android client, although the API could support one.
- **No payments or ticketing.** Unlike some commercial suites, ClubMS does not support paid events, dues collection, or budgeting workflows.
- **No email or push notifications.** State changes (approval, RSVP reminders, co-host invitations) are visible only when a user next opens the application; there is no outbound notification channel yet.
- **No institutional SSO/LDAP integration.** Authentication is self-contained; it does not integrate with an existing university identity provider.

- **Free-text department field.** Departments used for join-rule targeting are free text rather than a curated, centrally managed registry, which can lead to near-duplicate entries (e.g., inconsistent capitalization or abbreviation) if not disciplined by administrators.

7.3 Recommendations and Future Work

1. **Notifications.** Add email and/or in-app push notifications for membership decisions, upcoming RSVP'd events, and co-host invitations.
2. **Single sign-on.** Support institutional SSO (SAML/OIDC) or LDAP-backed authentication as an alternative to local email/password accounts.
3. **Multi-tenancy.** Extend the schema and authorization model to allow one deployment to host several independently branded institutions.
4. **Co-curricular transcript.** Aggregate a student's memberships, officer positions, and event participation into an exportable involvement record, mirroring a feature common in commercial suites [3].
5. **Native mobile client.** Build a companion mobile application against the existing REST API, prioritizing the student-facing browse/join/RSVP flows.
6. **Budgeting and ticketing.** Add optional financial workflows (dues, paid event tickets) for clubs and institutions that need them.
7. **Calendar interoperability.** Support ICS export/subscription so a student's joined-club events appear in their personal calendar application.
8. **Internationalization.** Externalize UI strings to support multiple languages beyond the current English-only interface.

Table 7.1: Limitations mapped to recommended future work

Limitation	Recommended future work
No notifications	Email / in-app push notifications
No SSO/LDAP integration	Institutional SSO or LDAP support
Single-tenant deployment	Multi-tenant architecture
No involvement transcript	Co-curricular transcript export
No native mobile application	Companion mobile client on the existing API
No payments/ticketing	Budgeting and paid-event ticketing module

REFERENCES

[1] S. H. Syd. Amra, "Online Club Management System for UiTM Terengganu (OnClubS)," B.Sc. thesis, Faculty of Computer and Mathematical Sciences, Universiti Teknologi MARA, Jan. 2017.

[2] Y. Ainapure, R. Patil, S. Bhadgaonkar, P. Sawant, and M. Pardesi, "DYPCETCLUBS – A Unified Platform for Management of College Clubs," *International Journal of Innovative Research in Computer and Communication Engineering*, vol. 13, no. 3, Mar. 2025.

[3] Modern Campus and CampusGroups, "Student engagement and involvement platforms," Modern Campus Involve / CampusGroups product documentation, accessed 2026. [Online]. Available: https://moderncampus.com/products/student-engagement-and-learning-platform and https://www.readyeducation.com/campusgroups/

[4] S. Kumar, S. Kumar, S. Chowdary, and E. Murali, "Club Management System: A MERN Stack-Based Web Application for Event Management in Colleges," *International Journal of Innovations in Engineering and Science*, vol. 10, no. 7, Jun. 2025.

[5] K. K. Nath, S. Mohite, S. Dhapate, S. S. Chauhan, and A. Gaikwad, "College Club Management System – Digitizing Student Engagement and Organizational Efficiency," *Iconic Research and Engineering Journals*, vol. 8, no. 11, pp. 1433–1437, May 2025.

[6] W. Qi, "Design and Implementation of University Club Management System based on WeChat Applet," *Journal of Computing and Electronic Information Management*, 2024.

[7] J. S. Park, R. Sandhu, and G.-J. Ahn, "Role-Based Access Control on the Web," *ACM Transactions on Information and System Security*, vol. 4, no. 1, pp. 37–71, Feb. 2001.

[8] W. A. Jansen, "A Revised Model for Role-Based Access Control," NISTIR 6192, National Institute of Standards and Technology, Jul. 1998.

[9] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[10] OWASP Foundation, "OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[11] Meta Platforms, Inc., "React Documentation," 2026. [Online]. Available: https://react.dev

[12] OpenJS Foundation, “Express.js Documentation,” 2026. [Online]. Available: https://expressjs.com

[13] Prisma Data, Inc., “Prisma ORM Documentation,” 2026. [Online]. Available: https://www.prisma.io/docs

[14] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” 2024. [Online]. Available: https://www.postgresql.org/docs/16/

[15] Docker Inc., “Docker Documentation,” 2026. [Online]. Available: https://docs.docker.com

[16] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[17] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *Proceedings of the USENIX Annual Technical Conference*, 1999.

[18] Tailwind Labs Inc., “Tailwind CSS Documentation,” 2026. [Online]. Available: https://tailwindcss.com/docs

[19] WorkOS, “Radix UI Primitives Documentation,” 2026. [Online]. Available: https://www.radix-ui.com

[20] C. McDonnell, “Zod: TypeScript-first Schema Validation,” 2026. [Online]. Available: https://zod.dev

[21] AtomicJar, Inc., “Testcontainers Documentation,” 2026. [Online]. Available: https://testcontainers.com